PROBLEM STATEMENT

In urban environments, the increasing number of vehicles has made finding an available parking space a significant challenge. This leads to unnecessary circling of parking lots, resulting in traffic congestion, increased fuel consumption, and higher pollution levels. Traditional parking systems lack real-time availability information, causing driver frustration and inefficiency.

The challenge is to design and develop an IoT-based system that can:

1. Automatically detect the occupancy status of multiple parking slots in real-time.
2. Provide immediate, local feedback on slot availability within the parking area.
3. Transmit this data to a cloud platform for remote monitoring and accessibility.
4. Offer a user-friendly mobile interface for users to check parking availability before arriving at the location.
5. Be cost-effective, scalable, and easy to implement using widely available components.

This project addresses these problems by creating a smart parking monitoring system using ESP32, IR sensors, an LCD, and the Blynk IoT platform.

INTRODUCTION

The Internet of Things (IoT) is revolutionizing various sectors, including urban planning and transportation. A Smart Parking System is a quintessential IoT application that enhances the efficiency of parking management. This project, "Car Parking Monitoring System using ESP32 and Blynk IoT," leverages this technology to provide a real-time solution.

The system utilizes Infrared (IR) sensors placed in each parking bay to detect the presence of a vehicle. The ESP32 microcontroller, known for its powerful processing and built-in Wi-Fi capability, acts as the central control unit. It reads data from all sensors, processes it to determine available slots, and displays a summary on a 16x2 LCD screen with an I2C module for easy interfacing.

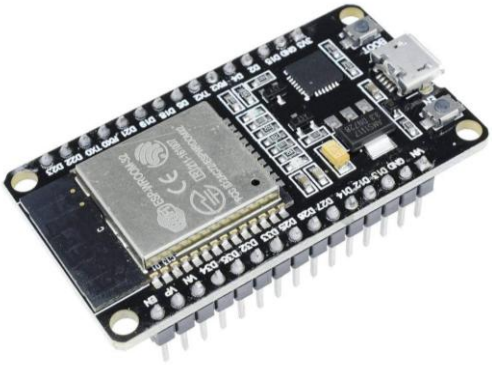
Furthermore, the ESP32 connects to the Blynk IoT platform over Wi-Fi. Blynk serves as the remote monitoring interface, where a custom dashboard with graphical widgets (like LEDs or gauges) shows the live status of each parking slot. This allows users to view parking availability from anywhere, reducing search time and congestion.

This project serves as a foundational model for understanding sensor networks, wireless data transmission, and cloud-based IoT application development, paving the way for more intelligent and connected cities.

KEY COMPONENTS

1. ESP32 Development Board

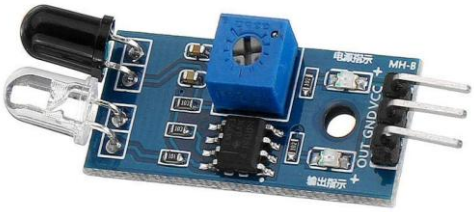
- **Function:** Serves as the main microcontroller for reading sensors, driving the display, and communicating with the Blynk cloud.



- **Key Features:**
    - Dual-core processor.
    - Integrated Wi-Fi and Bluetooth.
    - Sufficient GPIO pins for multiple sensors and peripherals.
- **Importance:** The brain of the project, handling all processing and connectivity tasks.

2. IR Sensor Module (x4)

- **Function:** Detects the presence of a vehicle in a parking slot by emitting and receiving infrared light.



- **Key Characteristics:**
    - Outputs a digital signal (LOW when an obstacle is detected).
    - Operating voltage: 3.3V - 5V.
- **Importance:** Acts as the primary input device to sense parking slot occupancy.

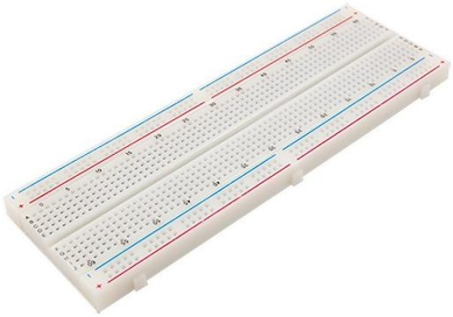
3. 16x2 LCD Display with I2C Module

- **Function:** Displays the real-time status of the parking system (e.g., "Slots Free: 2").



- **Key Features:**
    - 16 character x 2 line display.
    - I2C interface reduces wiring complexity.
- **Importance:** Provides local, on-site visual feedback for the system status.

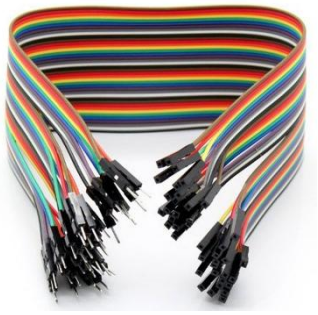
4. Breadboard



- **Function:** A platform for prototyping and making temporary connections without soldering.
- **Importance:** Allows for easy circuit building, testing, and modification.

5. Jumper Wires

- **Function:** Used to establish electrical connections between components on the breadboard and the ESP32.
- **Types Used:** Male-to-Male (M-M) jumper wires.
- **Importance:** Essential for creating a flexible and reusable circuit prototype.



BLYNK IOT APP AND ITS INTERFACING

The Blynk IoT platform is a cloud-based system that enables real-time monitoring and control of hardware devices through a smartphone or web dashboard. It is widely used in IoT applications for its simplicity and flexibility. Blynk operates through three core components: the Blynk App, the Blynk Cloud, and the hardware device (in this case, an ESP32). The ESP32 connects to Wi-Fi and communicates with the Blynk Cloud using an authentication token, allowing seamless data transfer between the hardware and the mobile app.

In this IoT-based car parking system, the Blynk app is used to display the status of parking slots remotely. Each parking slot is monitored by an IR sensor connected to the ESP32. When a car occupies a slot, the sensor detects it and sends a digital signal to the ESP32. The microcontroller processes this input and updates both the LCD display and the Blynk dashboard accordingly.

To set up the Blynk interface, a new project is created on the Blynk IoT app, where widgets such as LED indicators are added. Each LED is linked to a virtual pin (V0, V1). The ESP32 updates these virtual pins through commands like Blynk.virtualWrite(). When a slot is empty, the LED glows to indicate availability, and when occupied, the LED turns off. This provides a clear visual indication of parking status to users remotely.

The interfacing in this project connects multiple components:

- IR Sensors → ESP32 GPIO pins (32, 33) for vehicle detection
- LCD (I^2C) → ESP32 SDA and SCL pins for on-site status display
- ESP32 → Wi-Fi → Blynk Cloud → Mobile App for IoT-based monitoring

Through this setup, the system offers both local and remote visualization of parking availability. The LCD displays messages such as “Slot 1: Full” or “Slot 2: Empty,” while the Blynk app mirrors the same information via LEDs in real time.

In conclusion, the Blynk IoT app acts as a powerful interface bridging hardware and cloud. Its integration with the ESP32 and sensors makes this car parking system intelligent, efficient, and user-friendly, demonstrating the real-world potential of IoT in smart automation and monitoring applications.

CODE

```
#define BLYNK_TEMPLATE_ID "TMPL33TjDN84x"
#define BLYNK_TEMPLATE_NAME "IoT App"
#define BLYNK_AUTH_TOKEN "g7zWfM6ISqKo9havuQ7K_C21Piuo-MDD"

#include <Wire.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <LiquidCrystal_I2C.h>

// LCD setup -
LiquidCrystal_I2C lcd(0x27, 16, 2);

#define SENSOR1 32
#define SENSOR2 33

char auth[] = "g7zWfM6ISqKo9havuQ7K_C21Piuo-MDD";
char ssid[] = "GT";
char pass[] = "12345678";

BlynkTimer timer;

void readSensors() {
   int L1 = digitalRead(SENSOR1);
   int L2 = digitalRead(SENSOR2);

   // Sensor 1
   if (L1 == 1) {
      Serial.println("Sensor 1 - No Object");
      lcd.setCursor(0, 0);
```

```
    lcd.print("1:Empty     ");
    Blynk.virtualWrite(V0, 255); // Turn on LED widget
  } else {
    Serial.println("IR Sensor 1 detected - Object Present");
    lcd.setCursor(0, 0);
    lcd.print("1:Full      ");
    Blynk.virtualWrite(V0, 0); // Turn off LED widget
  }

  // Sensor 2
  if (L2 == 1) {
    Serial.println("Sensor 2 - No Object");
    lcd.setCursor(0, 1);
    lcd.print("2:Empty     ");
    Blynk.virtualWrite(V1, 255); // Turn on LED widget
  } else {
    Serial.println("IR Sensor 2 detected - Object Present");
    lcd.setCursor(0, 1);
    lcd.print("2:Full      ");
    Blynk.virtualWrite(V1, 0); // Turn off LED widget
  }
}

void setup() {
  Serial.begin(115200);

  // Initialize LCD with proper delays
  lcd.init();
  delay(100);
  lcd.backlight();
```

```
delay(100);

// Show welcome message
lcd.setCursor(0, 0);
lcd.print("  Welcome To  ");
lcd.setCursor(0, 1);
lcd.print(" Car Parking");
delay(3000);

// Clear and show connecting message
lcd.clear();
lcd.setCursor(0, 0);
lcd.print("Connecting to");
lcd.setCursor(0, 1);
lcd.print("WiFi...");

// Initialize Blynk with timeout
WiFi.begin(ssid, pass);

unsigned long startTime = millis();
while (WiFi.status() != WL_CONNECTED && millis() - startTime < 10000) {
  delay(500);
  Serial.print(".");
}

if (WiFi.status() == WL_CONNECTED) {
  Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("WiFi & Blynk");
```

```
    lcd.setCursor(0, 1);
    lcd.print("Connected!   ");
    Serial.println("\nWiFi and Blynk connected!");
  } else {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("WiFi Failed  ");
    lcd.setCursor(0, 1);
    lcd.print("Check Credentials");
    Serial.println("\nWiFi connection failed!");
  }

  // Setup sensor pins
  pinMode(SENSOR1, INPUT_PULLUP);
  pinMode(SENSOR2, INPUT_PULLUP);

  // Setup timer to read sensors every 500ms
  timer.setInterval(500L, readSensors);

  delay(2000);
  lcd.clear();
}

void loop() {
  Blynk.run();
  timer.run();
}
```

CODE EXPLANATION

This Arduino IDE program for the ESP32 microcontroller is the core logic for the Smart Parking System. It handles sensor data reading, local display output, and cloud communication.

1. Blynk Configuration

```
#define BLYNK_TEMPLATE_ID "TMPL33TjDN84x"
#define BLYNK_TEMPLATE_NAME "IoT App"
#define BLYNK_AUTH_TOKEN "g7zWfM6ISqKo9havuQ7K_C21Piuo-MDD"
```

These lines link your ESP32 with your specific Blynk project using a unique template ID, name, and authentication token.

2. Library Inclusions

```
#include <Wire.h>
#include <WiFi.h>
#include <BlynkSimpleEsp32.h>
#include <LiquidCrystal_I2C.h>
```

- Wire.h – Handles I²C communication between ESP32 and LCD.
- WiFi.h – Connects the ESP32 to your Wi-Fi network.
- BlynkSimpleEsp32.h – Enables communication with the Blynk IoT platform.
- LiquidCrystal_I2C.h – Allows easy control of the LCD through the I²C interface.

3. LCD and Sensor Setup

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
#define SENSOR1 32
#define SENSOR2 33
```

- The LCD is set to I²C address 0x27 with 16 columns and 2 rows.

- Two IR sensors are connected to GPIO 32 and 33.

4. Wi-Fi and Blynk Credentials

char auth[] = "g7zWfM6ISqKo9havuQ7K_C21Piuo-MDD";

char ssid[] = "GT";

char pass[] = "12345678";

These store your Blynk authentication token and Wi-Fi details.
The ESP32 uses these credentials to connect to the internet and Blynk server.

5. Timer Declaration

BlynkTimer timer;

The timer executes tasks at regular intervals — here, it repeatedly calls the readSensors() function every 500 ms.

6. Sensor Reading Function

```
void readSensors() {
  int L1 = digitalRead(SENSOR1);
  int L2 = digitalRead(SENSOR2);
```

Reads the logic level from each sensor:

- HIGH (1): No object detected → Slot Empty
- LOW (0): Object detected → Slot Full

Slot 1 Logic

```
if (L1 == 1) {
  lcd.setCursor(0, 0);
```

```
    lcd.print("1:Empty     ");
    Blynk.virtualWrite(V0, 255);
} else {
    lcd.setCursor(0, 0);
    lcd.print("1:Full      ");
    Blynk.virtualWrite(V0, 0);
}
```

Displays “1:Empty” or “1:Full” on the LCD’s first line.
Sends data to Blynk virtual pin V0 → LED widget ON (empty) or OFF (full).

Slot 2 Logic

```
if (L2 == 1) {
    lcd.setCursor(0, 1);
    lcd.print("2:Empty     ");
    Blynk.virtualWrite(V1, 255);
} else {
    lcd.setCursor(0, 1);
    lcd.print("2:Full      ");
    Blynk.virtualWrite(V1, 0);
}
```

Displays “2:Empty” or “2:Full” on the LCD’s second line.
Sends data to Blynk virtual pin V1 for the second LED widget.

7. Setup Function

```
void setup() {
```

```
Serial.begin(115200);
lcd.init();
lcd.backlight();
```

Starts serial communication for debugging and initializes the LCD.

Welcome Message

```
lcd.setCursor(0, 0);
lcd.print("  Welcome To  ");
lcd.setCursor(0, 1);
lcd.print(" Car Parking");
delay(3000);
```

Displays a welcome message for 3 seconds when the system powers up.

Wi-Fi Connection Process

```
lcd.clear();
lcd.print("Connecting to");
lcd.setCursor(0, 1);
lcd.print("WiFi...");
WiFi.begin(ssid, pass);
```

Shows connection progress on the LCD.
The ESP32 attempts to connect to Wi-Fi for up to 10 seconds.

If Connection Successful

```
Blynk.begin(auth, ssid, pass, "blynk.cloud", 80);
lcd.print("WiFi & Blynk");
```

lcd.setCursor(0, 1);

lcd.print("Connected!");

Displays success message on LCD and initializes Blynk communication.

If Connection Fails

lcd.clear();

lcd.print("WiFi Failed");

lcd.setCursor(0, 1);

lcd.print("Check Credentials");

Shows an error message if Wi-Fi credentials are incorrect or network is unavailable.

Sensor and Timer Setup

pinMode(SENSOR1, INPUT_PULLUP);

pinMode(SENSOR2, INPUT_PULLUP);

timer.setInterval(500L, readSensors);

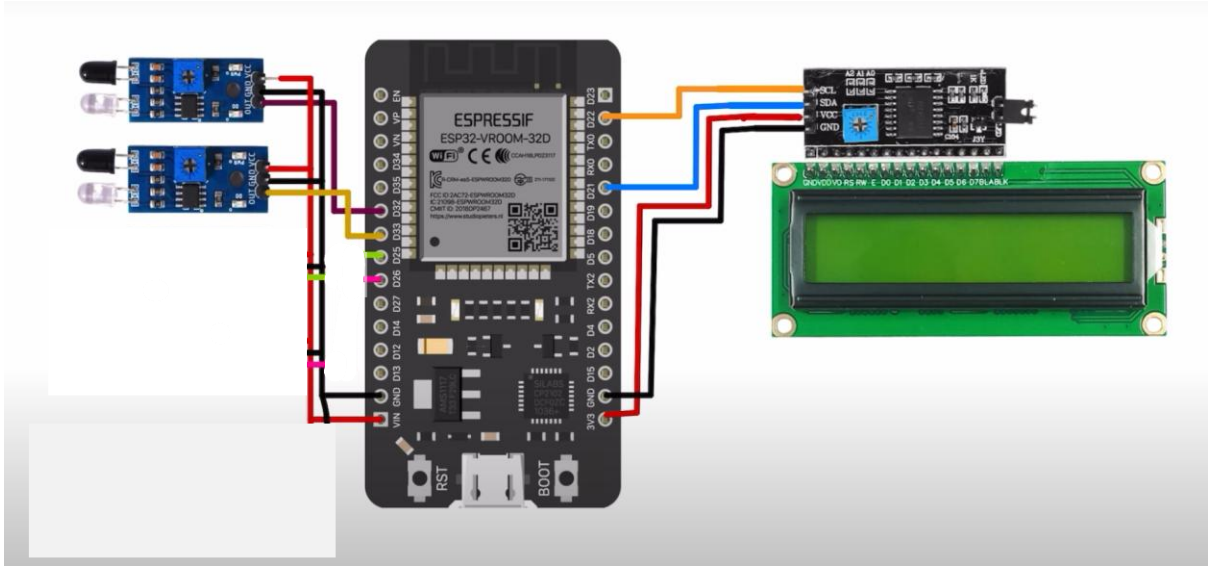
Configures both sensor pins as input with pull-up resistors and sets the readSensors() function to run every 500 milliseconds.

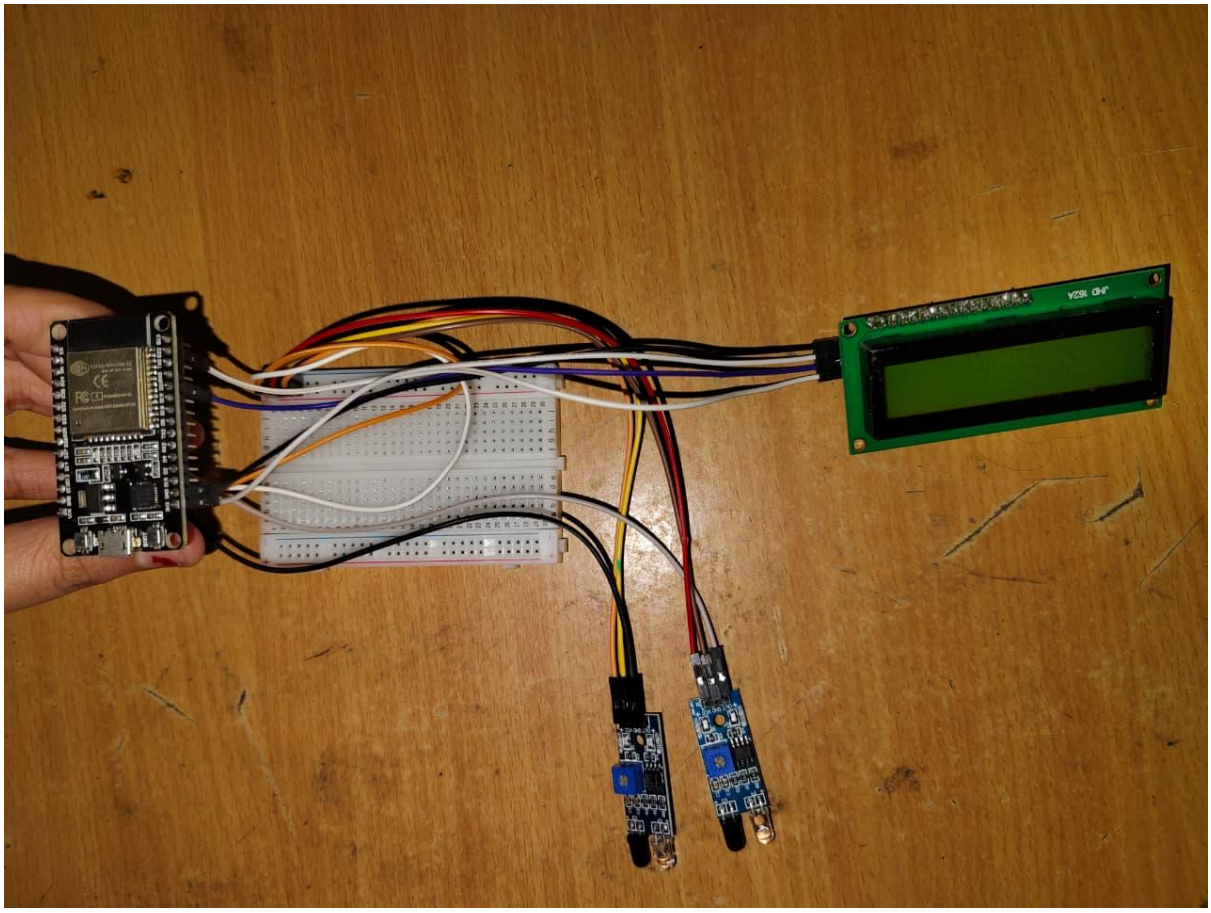
8. Loop Function

```
void loop() {
   Blynk.run();
   timer.run();
}
```

Blynk.run() maintains continuous communication with the Blynk cloud.
timer.run() executes the scheduled task (sensor reading).

CIRCUIT





RESULTS

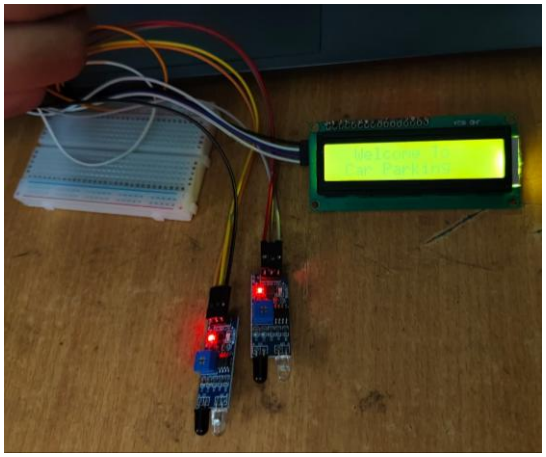


Figure 1:Welcome Message.

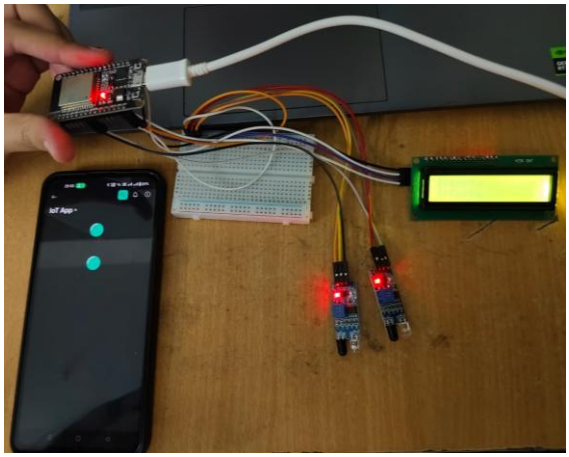


Figure 2:When parking is empty.

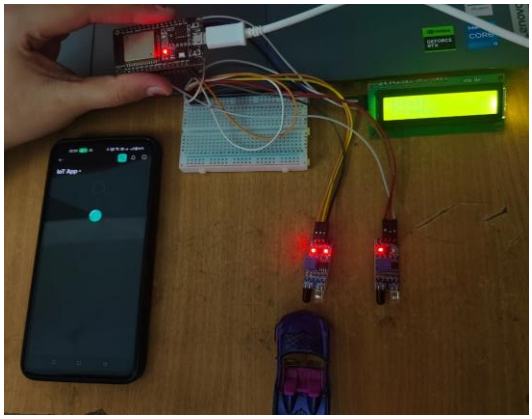


Figure 3:When Parking A is full and B is empty.

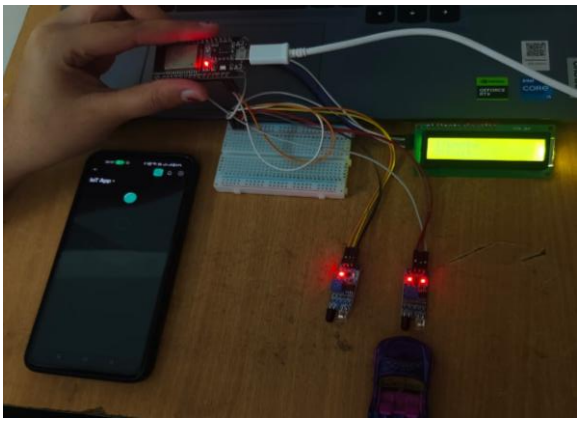


Figure 4: When Parking B is full and A is empty.

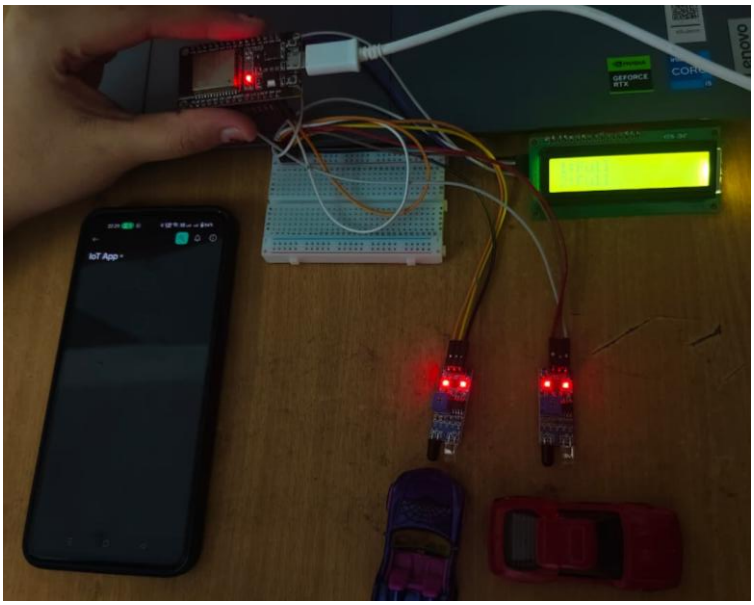


Figure 5:When both Parking A and B are full.

APPLICATIONS

The developed Car Parking Monitoring System has a wide range of practical applications in various domains:

1. **Shopping Malls and Multiplexes:** Helps direct customers to available spots, improving their experience and reducing traffic inside parking facilities.
2. **Airports and Railway Stations:** Manages the high volume of short-term and long-term parking efficiently, crucial for such high-traffic areas.
3. **Smart City Infrastructure:** Can be deployed as part of a larger city-wide smart parking solution, with data aggregated to guide drivers to lots with available space.
4. **Office Complexes and Corporate Campuses:** Allows employees and visitors to quickly find parking, saving time and reducing congestion during peak hours.
5. **Residential Apartments (Gated Communities):** Helps residents and their guests locate available parking slots within large apartment complexes.
6. **University and College Campuses:** Manages parking for students, faculty, and staff across large campus grounds.
7. **Hospitals:** Ensures that critical parking spaces, such as those for emergency vehicles or patient drop-off, are monitored and managed effectively.
8. **Public Event Venues:** Useful for managing parking during concerts, sports events, or fairs where parking demand is temporary and high.

ADVANTAGES

The proposed system offers several significant advantages over traditional parking methods:

1. **Real-Time Information:** Provides live updates on parking availability, eliminating the need for drivers to circle lots searching for a space.
2. **Reduced Traffic Congestion:** By minimizing search time for parking, it directly contributes to reduced vehicular congestion in and around parking areas.
3. **Fuel and Emission Savings:** Less time spent searching for parking leads to lower fuel consumption and a reduction in associated carbon emissions.
4. **Improved User Experience:** Enhances driver convenience and reduces frustration by providing clear, accessible information both on-site and remotely.
5. **Cost-Effective Solution:** Built using widely available, low-cost components like the ESP32 and IR sensors, making it an affordable automation solution.
6. **Scalability:** The system can be easily scaled to monitor more parking slots by adding more sensors and utilizing microcontrollers with more GPIO pins or multiplexing techniques.
7. **Remote Monitoring Capability:** The Blynk IoT integration allows for remote management and monitoring, enabling facility managers to oversee the parking lot from anywhere.
8. **Educational Value:** Serves as an excellent project for students to learn about IoT, sensor interfacing, microcontroller programming, and cloud connectivity.

DISADVANTAGES

1. **Line-of-Sight Requirement of IR Sensors:** IR sensors can be affected by environmental factors like direct sunlight, dust, and rain, potentially leading to false readings. They also require a clear line of sight.
2. **Limited Range:** The effective detection range of basic IR sensors is short, making them suitable only for close-range vehicle detection.
3. **Dependence on Network Connectivity:** The system's remote monitoring feature is entirely dependent on a stable Wi-Fi connection. Loss of internet renders the Blynk app useless.
4. **Blynk Service Dependency:** The system relies on the Blynk cloud service, which is a third-party platform. Changes to its API or service terms could affect the project.
5. **Limited Data Logging:** The basic implementation does not store historical data for analyzing parking patterns over time.
6. **Power Consumption:** While the ESP32 is efficient, continuous Wi-Fi operation and sensor polling drain the battery, requiring frequent recharging for truly wireless operation.

FUTURE SCOPE

1. **Advanced Sensors:** Replace IR sensors with ultrasonic sensors for better accuracy and range, or magnetic (reed) sensors for in-ground installation.
2. **Solar Power Integration:** Add a small solar panel to continuously charge the battery, making the system fully self-sustainable.
3. **Slot Reservation and Payment System:** Develop a companion mobile app that allows users to reserve a slot in advance and pay for it digitally.
4. **LED Guidance System:** Install Green/Red LEDs above each parking spot to visually guide drivers to the exact location of a free space.
5. **Alternative IoT Platforms:** Migrate the cloud logic to a more robust platform like AWS IoT or Google Cloud IoT for enhanced scalability and data analytics.
6. **Data Analytics Dashboard:** Create a web-based dashboard for parking lot operators to view usage statistics, peak hours, and revenue reports.

CONCLUSION

The "Car Parking Monitoring System using ESP32 and Blynk IoT" project was successfully designed, implemented, and tested. The system effectively demonstrates the integration of hardware components like the ESP32, IR sensors, and an LCD display with the Blynk IoT software platform to create a practical solution for a real-world problem.

The prototype reliably monitors the status of four parking slots, displays the availability locally, and transmits this data to a cloud-based mobile application for remote access. This project validates the core concept of using IoT to make parking management more efficient, user-friendly, and intelligent.

While the current system is a prototype, it provides a solid foundation. The knowledge gained in sensor interfacing, Wi-Fi communication, and IoT platform integration is invaluable. The identified future scopes, such as adding reservation systems and solar power, present exciting pathways for further development.

In conclusion, this project not only achieves its technical objectives but also highlights the transformative potential of IoT in building smarter urban infrastructures. It serves as a comprehensive learning experience and a stepping stone towards more advanced industrial-grade systems.

Course Outcomes:

CO 4: Ability to design and implement IoT-based solutions for real-world problems.

CO 5: Develop skills in interfacing sensors and actuators with microcontrollers like ESP32.

REFERENCES

1. **Blynk Official Documentation:** (n.d.). *Getting Started with Blynk*. Blynk.io. Retrieved from https://docs.blynk.io/

2. **Espressif Systems.** (2023). *ESP32 Datasheet*. Retrieved from https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

3. **Arduino.** (n.d.). *Arduino Reference*. Retrieved from https://www.arduino.cc/reference/en/

4. **TutorialsPoint.** (2023). *IoT Tutorial*. Retrieved from https://www.tutorialspoint.com/internet_of_things/index.htm

5. **Instructables.** (2022). *ESP32 with Blynk: A Step-by-Step Guide*. Retrieved from https://www.instructables.com/ESP32-With-Blynk/

6. **Arduino Blog.** (2023). *Getting Started with the LCD 1602 Display*. Retrieved from https://blog.arduino.cc/2020/06/22/getting-started-with-the-lcd-1602-display/